

AI 协作反思日志 #3 — Phase 3 详细设计

团队：BUG一改就队 设计小组 日期：2026年5月10日

1. 本阶段 AI 使用清单

任务	AI工具	Prompt摘要	AI输出质量(1-5)	人工修改幅度(%)
核心类图初稿生成	GitHub Copilot & 通用大模型	根据 Phase 3 需求输出覆盖 P0 功能的类图草案，要求包含类、属性、方法和关系	3	45
SOLID 缺陷审查	GitHub Copilot	对 AI 生成的类设计逐条按 SOLID 原则检查，指出问题并给出修正方向	4	30
API 文档初稿生成	Gemini	基于用户、任务、订单、评价流程生成 RESTful API 规范草案	4	35
数据库表结构与索引建议	GitHub Copilot & DeepSeek	根据业务流程生成建表 SQL，并分析高频查询索引和隐私数据处理方式	3	40
详细设计文档整合	GitHub Copilot	将类设计、API 设计、ER 设计、索引与隐私方案整合为统一详设文档	4	25

2. AI 助力分析（Q1）

本阶段 AI 在“从需求到设计草稿”的环节提供了明显帮助，主要体现在以下几个方面。

第一，AI 能快速把需求中的功能点映射成设计对象。团队在做 Phase 3 时，需要同时处理类图、接口、数据库三类设计产物。AI 在短时间内给出了用户、任务、订单、评价、通知等核心对象的初始结构，减少了从零搭框架的时间。

第二，AI 在接口文档和数据库初稿上效率较高。对于用户注册、登录、发布任务、浏览任务、抢单、查看订单详情、提交评价等标准流程，AI 能较快整理出 URL、HTTP 方法、请求参数和响应格式，也能产出基础的建表 SQL。这使团队可以把更多精力放在一致性检查和业务约束补充上，而不是重复写模板。

第三，AI 适合做“覆盖性检查”。在数据库和 API 设计中，AI 会提醒我们补充错误码、认证要求、分页约束、唯一索引、敏感字段保护等内容。虽然这些建议并不总是完全准确，但能帮助团队减少明显遗漏。

总体来看，AI 在本阶段最有价值的角色不是“替代设计”，而是“快速生成可讨论的第一稿”和“辅助检查是否漏项”。

3. AI 误导分析（Q2）

SOLID 检查中发现的 AI 设计问题数量：5

最严重的设计问题是：AI 初版把大量业务能力堆进一个类似 CampusHubManager 的大类中，同时又给出一个过大的 CampusHubService 接口，导致职责边界模糊，后续扩展时几乎每加一个需求都要改动已有核心类。

本阶段 AI 的误导主要有三类。

第一类是职责聚合过度。AI 生成的初稿表面上“功能都覆盖了”，但实际上把用户管理、任务发布、订单流转、评价、通知都塞进一个服务类里。这种方案短期看起来完整，长期会直接破坏可维护性。

第二类是过度依赖条件分支。无论是订单状态流转，还是任务分类处理，AI 倾向于用 if-else 或 switch 快速拼接逻辑，而不是主动抽象状态模式或策略模式。这样生成的设计虽然能解释流程，但扩展能力很差。

第三类是工程约束考虑不足。AI 在数据库和接口生成时，容易忽视认证边界、幂等控制、唯一约束、索引的读写平衡以及实名信息加密等问题。也就是说，AI 更擅长“描述业务表面”，不擅长“替团队做工程取舍”。

因此，本阶段最大的经验是：AI 给出的设计看起来越完整，越要重点检查它是否隐藏了高耦合、高分支和低扩展性的问题。

4. Prompt 改进计划（Q3）

上阶段改进措施的验证结果：上阶段我们尝试把 Prompt 写得更具体，例如要求 AI 按模块输出、补充错误码、标出主外键和索引建议。结果证明这种做法有效，AI 的输出比之前更完整，遗漏字段和接口的情况明显减少。但它仍然会在职责划分和可扩展性上给出“看起来合理、实际不稳”的方案，因此仅靠“写得更详细”还不够。

本阶段新的改进措施：

1. 在 Prompt 中显式要求按“领域对象、应用服务、仓储抽象、状态对象、策略对象”分层输出，而不是只让 AI 给一个“完整类图”。
2. 对 API 和数据库设计增加约束词，例如“必须包含鉴权要求、错误码、分页约束、唯一索引、隐私字段保护策略”。

3. 要求 AI 在输出设计后，再额外输出“可能违反 SOLID 的点”和“后续最可能返工的位置”，把它从单纯生成工具变成自检工具。
4. 对 AI 产出的每一份初稿，都增加人工交叉核对步骤，尤其检查状态流转、权限控制、数据一致性和敏感信息处理。

5. 本阶段的核心工程判断

决策内容：订单流转不采用多个订单子类继承的方式建模，而采用 ServiceOrder 持有 OrderState 的状态模式设计，同时将匹配规则和信用规则抽象为策略接口。

为什么 AI 无法做这个决策：因为这个决策本质上不是“能不能生成代码或图”的问题，而是“要为后续演进成本承担什么代价”的问题。AI 可以给出几种表面上都能运行的方案，但它无法真正站在项目后续维护、团队协作、测试隔离和答辩表达的角度，对哪一种设计更稳做出可靠判断。这个判断依赖对项目范围、课程要求、未来变更成本以及团队实现能力的综合权衡，必须由人来拍板。